

Q Search



Gregor Ojstersek

@gregorjstersek



How to Do Spec-Driven Development

1

4

9

770

The workflow AI-native teams use to build complex software. This is how they do it!

Intro

Earlier this year, I was in San Francisco talking with different companies, and one of them was Larridin. Funnily, I came right in the middle of their team meeting (sorry guys).

They are building a platform for what many companies are wondering these days → how to measure AI adoption and AI fluency.

Things that especially stood out from our conversations:

- They are a true definition of an AI-native team, with AI being at the core of everything they do.
- They are big advocates of spec-driven development. The spec needs to be almost perfect before they start with the implementation.
- Every engineer owns the feature end-to-end → from creating the spec to implementing it, to ensuring it's working correctly.

Relevant people

Gregor Ojstersek

@gregorjstersek

Follow

CTO | Founder/Author of Engineering Leadership newsletter (195k+ readers)
[newsletter.eng-leadership.com](#) | Helping you become a great engineering leader!

What's happening

- Politics · Trending
Vladimir Putin
- Only on X · Trending
#GodMorningFriday
- Trending in India
#जय_माता_दी
- Trending in India
#VinobaBhave

Show more

Terms · Privacy · Cookies · Accessibility ·
Ads Info · More ... © 2026 X Corp.





- [Ameya Kanitkar, CTO, Larridin](#), mentioned they also like to work with more junior-level engineers, but they need to leave and breathe AI and always look to improve and get better in this area.
- A great analogy about using AI → You have a fast-driving car right in front of you, why not use it?

I've talked with the devs, and also with Ameya, whose insights I am sharing today. In today's article, we'll dive into how they are doing spec-driven development at Larridin.

Let's start!

What is spec-driven development?

There are many different definitions out there, but here is what it means for me:

Spec-driven development is a way of building software where you focus on building a good specification first, and then use AI to implement the project.

There are many ways you can write the spec, for example, Michael Bolin, Codex Tech Lead, uses Notion to create a spec and then asks all the other engineers on the team to take a look and provide comments.

This helps spot any issues beforehand and improves the overall implementation done by the LLMs. To learn Michael's full process for AI-assisted engineering, read this [article](#).

The popularity behind spec-driven development is increasing

I am seeing spec-driven development as the way to build software used by more and more companies, and it definitely makes sense.

Especially with the cost of tokens just keep increasing, you don't have the luxury of just infinitely prompting in order to get the desired output (unless you work with unlimited tokens).

With a great spec, you can usually use a lower-tier model and get a very similar output, the reason for that is that the instructions in a great spec

Relevant people

Gregor Ojstersek

Follow

@gregorjstersek
CTO | Founder/Author of Engineering Leadership newsletter (195k+ readers)
newsletter.eng-leadership.com |
Helping you become a great engineering leader!

What's happening

Politics · Trending
Vladimir Putin ...

Only on X · Trending
#GodMorningFriday ...

Trending in India
#जय_माता_दी ...

Trending in India
#VinobaBhave ...

[Show more](#)

[Terms](#) · [Privacy](#) · [Cookies](#) · [Accessibility](#) ·
[Ads Info](#) · [More ...](#) © 2026 X Corp.



are very clear.

So, this is also the workflow that I see many companies use:

- Write a spec (with the help of higher-tier LLMs)
- Build the spec (with the lower-tier LLMs)

Now, let’s get into more details on how Larridin does spec-driven development!

1. Start with a spike first

Ameya mentioned that before they start writing the spec, they prove that the “risky” things work.

In software development, there are a lot of unknowns when you are building things, and it’s impossible to write a complete spec if you are dealing with unknowns.

So, what they do is, they create a spike task to resolve these unknowns of a specific project first.

The goal is not to build a full MVP, it can be a small, ugly, throwaway-quality work. Just enough to answer the question “does this actually work”?

The purpose is not to extend it afterward, as you would normally do with the MVP, but the goal is to remove any technical uncertainty, which, alternatively, would just contaminate the spec with various different options.

Some examples:

- If the API that you’re going to build has 3 edge cases, spike those.
- If your whole system depends on how fast your API is, spike the hot path and measure it.
- If you’re building a new view that has a lot of records in it, prove that the API can return the results fast enough.

And another example that Ameya shared from their work:

We wanted to create a model to measure “AI Fluency”. So we wrote a spike to ensure our core logic works. Once we were happy with the results, the rest of the design followed (queue, monitoring, database, etc.). Spec was simply for ensuring core logic works and is token efficient.

The goal is to stop when you have an answer, not when the code is pretty. If you’re worried about naming variables, functions, and file structure, stop, that’s a sign you’re putting too much thought into implementation details.

Relevant people

Gregor Ojstersek

@gregorjstersek

CTO | Founder/Author of Engineering Leadership newsletter (195k+ readers)

[newsletter.eng-leadership.com](#) | Helping you become a great engineering leader!

Follow

What’s happening

- Politics · Trending
Vladimir Putin
- Only on X · Trending
#GodMorningFriday
- Trending in India
#जय_माता_दी
- Trending in India
#VinobaBhave

Show more



2. Promote the spike to reference implementation

Once the spike works, don't throw it away. It's now your reference implementation.

This is now a reference that you can point the LLM to and say "I know this pattern works, because here it's working!"

Ameya believes that this is one of the most underrated things when you're doing spec-driven development. A working example is worth a lot more than a full page in the spec. That's how the model stops hallucinating and stops "guessing" the behavior as well.

The reason is that the model can now check the reference and see the approach that has been taken to implement that specific part.

The reference implementation belongs in the repo in a known location. All of the references are in a location: `spec/spikes/reference/referenceName`.

And also another tip here is to comment on the specific parts that prove the approach, and also add comments to the parts where things are hardcoded, maybe the error handling is missing, or the implementation is not on point.

The more info you give in the codebase, the more accurate the overall implementation by the LLM is going to be.

Ameya believes that a spike without comments can be similarly dangerous as not doing it at all, because the model can just reproduce the wrong parts alongside the right ones.

3. Write the spec

Now that you have the reference implementation with comments, it's time to write the spec! The spec should be detailed enough to hand to a junior engineer or a small model.

You can and should, of course, use AI to help you write the spec. When you use AI to write the spec, Ameya suggests forcing the model to restate what needs to be done, list any assumptions, and also flag any kind of ambiguity.

So, in case there are any non-handled assumptions and ambiguity left, you can either create a new spike or update the existing one.

Here is also a desired spec structure that they use at Larridin:

- **Problem statement at the top**

What you're solving and for whom. One paragraph.

- **Non-goals**

Relevant people

Gregor Ojstersek 
@gregorjstersek

Follow

CTO | Founder/Author of Engineering Leadership newsletter (195k+ readers)
newsletter.eng-leadership.com | Helping you become a great engineering leader!

What's happening

Politics · Trending
Vladimir Putin ...

Only on X · Trending
#GodMorningFriday ...

Trending in India
#जय_माता_दी ...

Trending in India
#VinobaBhave ...

[Show more](#)

[Terms](#) · [Privacy](#) · [Cookies](#) · [Accessibility](#) · [Ads Info](#) · [More ...](#) © 2026 X Corp.



What this system explicitly will not do. This is where you head off half the overengineering before it starts.

• Assumptions

The details that you have already decided, so the model doesn't "silently" pick them. Cursor or offset pagination? Lazy loading of components? You need to write all of these down.

• Reference implementation

Link to the spike. Call out (if not already in the comments) which parts are shortcuts, and which parts prove the implementation.

• Architecture

Data model, module boundaries, interfaces, error handling, etc.

• Test plan

What unit, integration, and/or e2e tests are needed and how they need to be done. More on this in the next phase.

Example of a good spec shared by Ameya: [Full example of the spec](#).

And a recommendation he shared:

My recommendation is to write the smallest spec that unambiguously specifies the system. If a section feels like fluff, cut it. If a decision feels obvious, write it down anyway. "Obvious to you" is not the same as "obvious to the model".

4. Write/adjust the test plan before the implementation plan

Test-driven development and spec-driven development are made for each other. Once you know what the system does, you can write the test plan before the implementation plan, not after.

The test plan is part of the spec, and you should go over it and adjust if something is not right or something is missing.

It's important that your test plan uses either bullet points or that the separation between tests is clear. And also you need to watch out for any "ambiguous tests".

Here are 2 examples:

- Instead of "Handles large inputs gracefully" it should be "processes 10k rows in under 2 seconds with memory under 500MB."
- Instead of "Fails safely on bad input" it becomes "returns a 400 with a specific error code when the payload is missing the customer_id field."

Relevant people

Gregor Ojstersek 
@gregorjstersek

Follow

CTO | Founder/Author of Engineering Leadership newsletter (195k+ readers) newsletter.eng-leadership.com | Helping you become a great engineering leader!

What's happening

Politics · Trending
Vladimir Putin ...

Only on X · Trending
#GodMorningFriday ...

Trending in India
#जय_माता_दी ...

Trending in India
#VinobaBhave ...

Show more

Terms · Privacy · Cookies · Accessibility ·
Ads Info · More ... © 2026 X Corp.



There's no need to implement the tests yet, it's important that you list them, name each one, and specify inputs and expected outputs.

That list should be a clear definition of done, which the implementation plan will be measured against.

We are going through the implementation plan in the next step!

5. Create the implementation plan

We now have the reference implementation ready, the spec is ready, and the test plan is on point. Now, it's time to go to the last step, which is creating the implementation plan.

The implementation plan is the boring part. A sequence of steps that turns the spec into code. File by file, function by function, each step is clearly defined, so there is no chance for an LLM to go a different direction.

This is much more detailed than a spec. If the spec is around 800 lines, then the implementation plan would be around 3k lines.

So, all you need to do here is ask an LLM to create an implementation plan based on the spec, and output it in a separate folder. You can have a separate folder in your repo called *plans*.

Once the plan is created, it's important that you do a detailed review. If you find that the implementation plan is making a new architectural call mid-implementation, that's a signal the spec wasn't complete.

Make sure to go back and update it, don't just manually update the implementation plan with architecture details and leave the spec unchanged.

The whole point of this workflow is that you stop making decisions in phase 3, where you wrote the spec. Making new ones in the last phase can silently introduce problems.

Here is also an example of a good implementation plan: **Full example of the implementation plan**. And you can find many other examples in the same folder.

Relevant people

Gregor Ojstersek

@gregorjstersek

CTO | Founder/Author of Engineering Leadership newsletter (195k+ readers)
[newsletter.eng-leadership.com](#) | Helping you become a great engineering leader!

Follow

What's happening

- Politics · Trending
Vladimir Putin
- Only on X · Trending
#GodMorningFriday
- Trending in India
#जय_माता_दी
- Trending in India
#VinobaBhave

Show more



6. Implement

So once you are happy with the implementation plan, now comes the easy part. You can use any of the small LLMs to do that. Even Haiku works great.

All you need to do is tell the LLM to create the code based on the implementation plan. It's your choice if you want to create the PR yourself, or you want to split the project into separate PRs, or tell an LLM to create the PRs, based on a specific convention, etc.

Once you have the implementation plan, the implementation is the easy part.

When to use spec-driven development (and when not to)

Spec-driven development is definitely an overhead for smaller changes. If you are doing a small change, like refactoring a specific file or a small adjustment of a feature, it's definitely not the right way to do it.

Prompt → review the code is the right way to go for these small changes.

But when you are working on a bigger project, and especially if you don't know if it's even feasible in the first place, it can save you a LOT of time rewriting and reviewing bad AI-generated code.

And also making "decisions on the fly" is usually a lot worse than making decisions beforehand. Especially if you are making important architecture decisions. Creating the spec forces you to really think if this is the right approach.

If you just prompt and adjust as you go, then you don't really think through whether the approach is correct or not.

Last words

Let's end this article with the following:

As you can see from the process they use at Larridin, clear thinking and good judgment are the most important parts. Once you've done that, writing the code is the easy part. Always keep this in mind.

Special thanks to the Larridin team for the hospitality and Ameya for their insights on how they work!

1

4

9

15



Relevant people

Gregor Ojstersek 
@gregorjstersek

Follow

CTO | Founder/Author of Engineering Leadership newsletter (195k+ readers)
newsletter.eng-leadership.com | Helping you become a great engineering leader!

What's happening

Politics · Trending
Vladimir Putin ...

Only on X · Trending
#GodMorningFriday ...

Trending in India
#जय_माता_दी ...

Trending in India
#VinobaBhave ...

Show more

Terms · Privacy · Cookies · Accessibility ·
Ads Info · More ... © 2026 X Corp.

